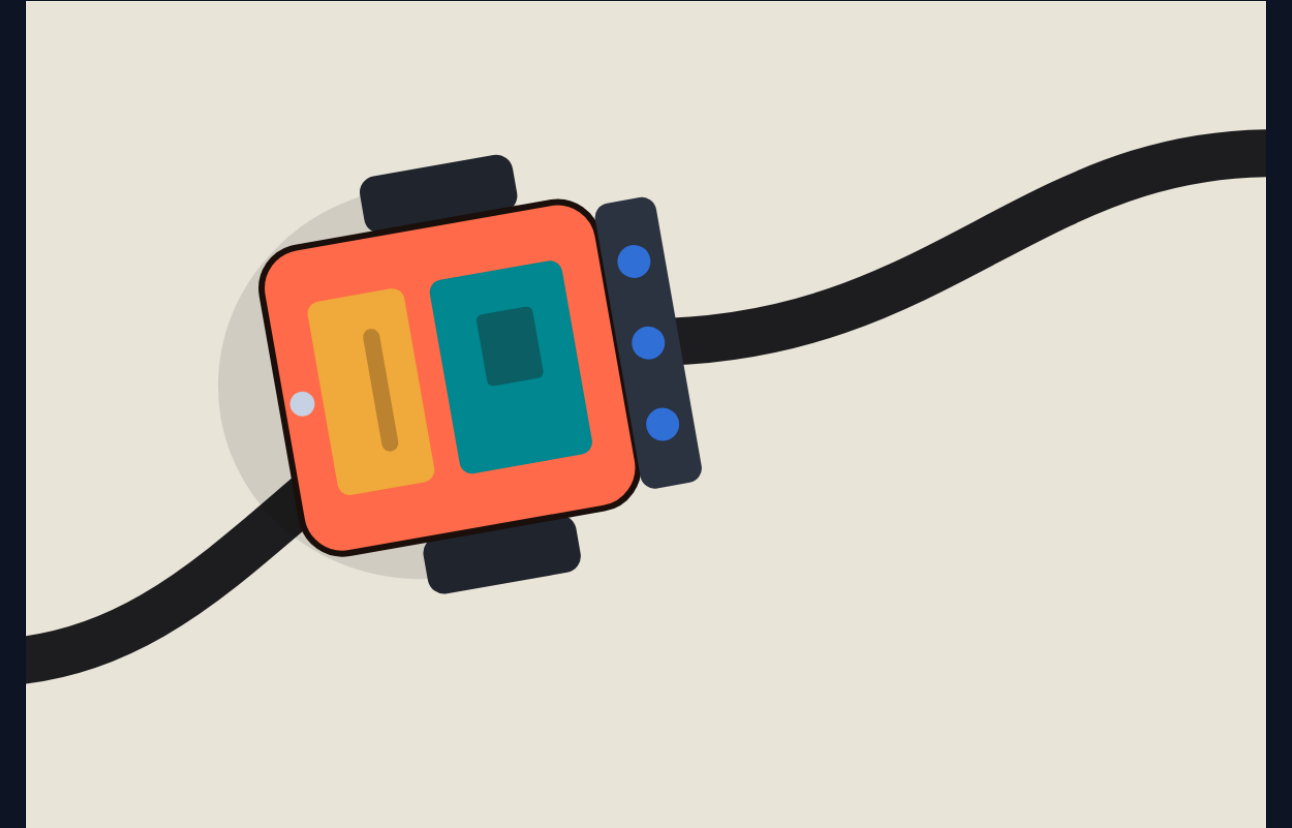


Robot dün çalışıyordu

Bugün çizgiyi kaybediyor. Teslim yaklaşıyor.

Arif Solmaz
İSTÜN Mekatronik Mühendisliği



Elif + Deniz · Bir robot · Ortak karar arayışı

Hangi dosyaya güveneceğiz?

Elif başka, Deniz başka bir “son sürüm” gönderdi.

SORUN

Hangisi son hali?

🕒 23:47

Robot Projesi


robot.ino


geçmiş: her değişiklik, kim, ne zaman

Robot Takımı

K

Kaan

Elif, son kodu atar mısın?

E

Elif

robot_final.ino'yu attım

robot_final.ino

D

Deniz

Ama ben sensörü robot_v2'de düzeltmiştim...

K

Kaan

Robot yine çizgiden çıktı. Hangisi son hali??

E

Elif

Bilmiyorum. Hepsini tek tek mi deneyelim?

GitHub'da tek bir **robot.ino** var; eski hâlleri geçmişinde saklı.

“SON” yazmak, geçmişi kaydetmek değildir.

Dosya var. Karar nerede?

Takımın eksikliği kod miktarı değil, değişikliğin izi.

Elimizde var

Kod dosyaları, birkaç kopya ve iki kişinin hatırladıkları.

Bulmamız gerekiyor

Ne değişti? Neden değişti? Hangi hâlden devam edeceğiz?

İlk hedef: hatırlamak yerine karşılaştırmak.

01 · ROBOTUN İZİNİ BUL

Dünle bugünü nasıl karşılaştırırız?

Geçmişte bir iz var: sensör eşiği 500 → 900.

2 FAYDA

Zaman makinesi: her değişiklik kayıtlı

E Elif'in ekranı

github.com/elif/robot-projem/commit/7f3e2a1

elif / robot-projem Public

<> Code Issues 0 Pull requests 0 Actions

Sensör eşiği değiştirildi

deniz committed yesterday 7f3e2a1

1 file changed

robot.ino +1 -1

3 int VIRAJ_HIZI = 120; // virajda hız

4 - int ESIK = 500; // sensör eşiği

4 + int ESIK = 900; // sensör eşiği

5

6 void setup() {

<> Browse files

SENİN İÇİN

Proje için zaman makinesi

✓ Hiçbir değişiklik kaybolmaz.

✓ Kim, ne zaman, neyi değiştirdi: hepsi kayıtlı.

✓ Bozmaktan korkma: her kayıt bir geri dönüş noktası.

Commit = bir kayıt anı, yanında kısa bir not.

Buldun: eşik 500'den 900'e çıkmış. O günkü hâline dönüp bakabilirsin.

Değişikliği bulduk. Doğru eşiği henüz bilmiyoruz.

Bu işin hangi tarafı Git?

Geçmiş bilgisayarda tutulur. Ortak çalışma GitHub'da görünür olur.

Git • yerel geçmiş

Değişiklikleri kaydeder. Hangi hâlden geldiğimizi gösterir.

GitHub • ortak çalışma

Depoyu barındırır. İşleri, önerileri ve incelemeyi bir araya getirir.

Git, GitHub dışındaki Git sunucularıyla da kullanılabilir.

01 · ROBOTUN İZİNİ BUL

Neyi birlikte saklayalım?

Robot kodu tek başına projeyi anlatmaya yetmiyor.

1 DAKİKA

Projeni GitHub'a koy

E Elif'in ekranı

github.com/elif/robot-projem

elif / robot-projem Public

<> Code Issues 0 Pull requests 0 Actions

main Add file <> Code

E elif Proje dosyaları eklendi 2 Commits

fotograflar Proje dosyaları eklendi

README.md Initial commit

devre-semasi.png Proje dosyaları eklendi

proje-raporu.pdf Proje dosyaları eklendi

robot.ino Proje dosyaları eklendi

README

robot-projem

Çizgi izleyen robot — ders projesi

SENİN İÇİN

Sadece kod değil: projenin her şeyi

✓ Arduino kodu

✓ Devre şeması

✓ Proje raporu

✓ Fotoğraflar

Repository (depo) = projenin internetteki klasörü.

Bitti: kod, devre şeması, rapor ve fotoğraflar artık internette.

Depo = proje dosyaları + kayıtlı geçmiş

0 satırda ne değişmiş?

Aynı değişkenin önceki ve sonraki değerini görüyoruz.

```
robot.ino
```

```
- const int ESIK = 500;  
+ const int ESIK = 900;
```

– önceki satır · + sonraki satır · commit: kaydedilmiş bir hâl

900 yanlış mı?

Git değişikliği gösterir. Doğru eşiği robotta doğrularız.

- Geçmiş: ne değişti?
- Doğrulama: hangi koşulda nasıl davranıyor?
- Karar: hangi kanıta dayanarak kabul edeceğiz?

500 ve 900, bu kurgusal senaryonun örnek değerleridir.

01 · ROBOTUN İZİNİ BUL

Bu sorunu kim takip ediyor?

İş kaydı: “IR sensör eşiğini incele.”

3 FAYDA

Takımın yapılacaklar listesi

E Elif'in ekranı

github.com/elif/robot-projem/issues

elif / robot-projem Public

<> Code Issues 2 Pull requests 0 Actions

Q is:issue state:open Labels New issue

2 Open 1 Closed Author Labels Assignee

Rapor için robotun fotoğrafını çek rapor #3 opened 2 minutes ago by elif

Motor sürücüsü ısınıyor bug #1 opened 2 days ago by kaan

SENİN İÇİN

Kim neyi yapıyor, herkes görür

İşler mesaj grubunda kaybolmaz; aylar sonra bile bulunur.

Her işin bir sorumlusu var.

Bitirme projesi, kulüp, ödev grubu: hepsine uyar.

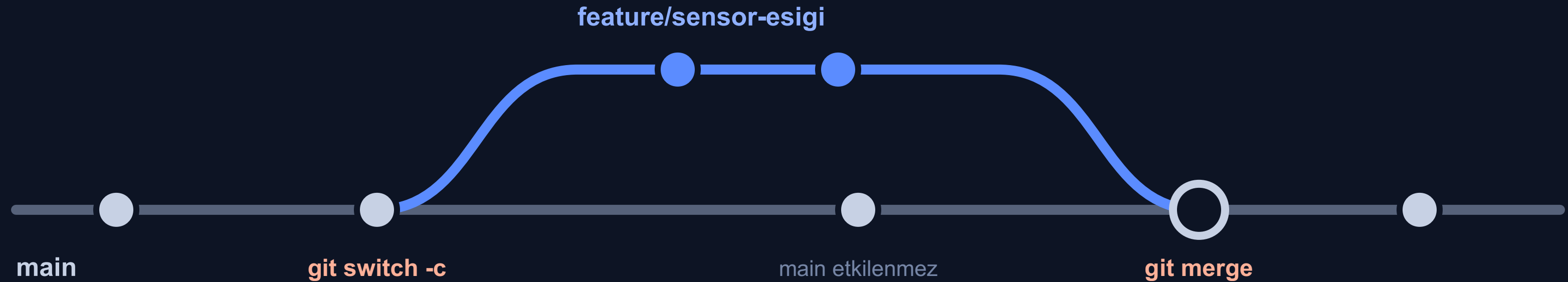
Issue = yapılacak bir iş ya da bir hata notu.

İş bitince **Close issue** : listeden düşer, kaydı kalır.

GIF: fotoğraf işi üzerinden issue arayüzü. Hikâyemizdeki iş: sensör eşiği.

Main aynı kalırken nasıl deneriz?

Aday çözümü ayrı bir dalda geliştireceğiz.



Dal açmak, değişikliği doğru ilan etmek değildir.

Dosyayı kaydettim. Deniz gördü mü?

Henüz değil. Düzenleme, kayıt ve paylaşım ayrı adımlar.



Düzenle → add → commit → push · Takımın yeniliğini almak: pull

Aynı başlangıcı nasıl alırız?

Elif projeyi dosyaları ve geçmişiyle bilgisayarına alıyor.

```
git clone <depo-adresi>  
cd robot-projem
```

Clone geçmişi getirir. ZIP, Git geçmişini oluşturmaz.

Deneme hangi dalda yapılacak?

Elif, sensör işi için ayrı bir çizgi açıyor.

```
git switch -c feature/sensor-esigi  
git branch
```

Etkin dal: feature/sensor-esigi · main bu işlemle değişmez.

Hangi çözümü deniyoruz?

900 → 500 dönüşünü aday değişiklik olarak hazırlıyoruz.

```
robot.ino
```

```
- const int ESIK = 900;  
+ const int ESIK = 500;
```

Kaydet → git status · Geçmiş ve GitHub henüz değişmedi.

Bu kayda ne girecek?

Sensör değişikliğini seçiyoruz. Diğer işler bekleyebilir.

```
git add robot.ino  
git diff --cached
```

Add seçer. Henüz yeni commit yok. GitHub'a gönderim yok.

Ne yaptığımızı nasıl kaydederiz?

Bir commit, değişikliği ve amacını geçmişte tutuyor.

```
git commit -m "IR sensör eşiği için 500 dönüş aday1"  
git log --oneline
```

Kayıt yerelde oluştu. “Aday” mesajı, doğrulama beklediğini anlatıyor.

Deniz şimdi görebilir mi?

Push sonrası aday çözüm GitHub'daki iş dalında görünür.

```
git push -u origin feature/sensor-esigi
```

Paylaşılmış aday ≠ main'e kabul edilmiş çözüm

Bu çözümü main'e alalım mı?

Pull request, takımın değerlendireceği değişiklik önerisi.

```
feature/sensor-esigi → main
```

Sorun: Robot çizgiyi kaybediyor.

Aday: ESİK, 900 → 500.

Gerekçe: Önceki değere dönüşü incele.

Doğrulama: Ölçüm ve davranış kaydı bekleniyor.

Issue: neye ihtiyacımız var? · PR: bu öneriyi kabul ediyor muyuz?

Deniz neye bakacak?

“Satır değişmiş” yeterli değil. Gerekçe ve davranış da anlaşılmalı.

- Neden bu değer?
- Hangi koşulda doğrulanacak?
- Başka bir davranışı etkiliyor mu?

Geri bildirim → aynı dalda yeni commit → güncellenen PR

Aynı satıra iki karar gelirse?

Elif'in adayında 500, gelen main'de 700 var.

```
robot.ino
```

```
<<<<<< HEAD
```

```
const int ESIK = 500;
```

```
=====
```

```
const int ESIK = 700;
```

```
>>>>>> main
```

İş dalındayız. main'i bu dala birleştiriyoruz. Git doğruluğa karar vermez.

Hangisini seçiyoruz?

Elif ve Deniz bu örnekte 500 adayını sürdürmeye karar veriyor.

```
robot.ino  
  
const int ESIK = 500;
```

1. İki değişikliğin gerekçesini birlikte değerlendiriyorlar.
2. Çakışma işaretlerini kaldırıp seçtikleri çözümü kaydediyorlar.
3. Doğrulamanın ardından aynı PR üzerinden incelemeyi tamamlıyorlar.

İşaretlerin kalkması, robotun sorununun çözüldüğünü kanıtlamaz.

Karar ortak geçmişe nasıl girer?

İnceleme ve doğrulama yeterliyse öneri main'e alınır.

Kabulden önce

İş dalında bir aday. Tartışılabilir ve değişebilir.

Kabulden sonra

Ortak main'de bir karar. Gerekçesi projede bulunabilir.

Görünür olmak → incelenmek → kabul edilmek

GitHub güncel. Bilgisayarım da mı?

Yerel main'i ayrıca güncelliyoruz.

```
git status  
git switch main  
git pull --ff-only
```

Push paylaşır. Pull getirir. Pull request birleştirme önerir.

Yeni biri robota nereden başlar?

Kodun yanına çalıştırmayı anlatan bir açıklama bırakıyoruz.

```
# Çizgi İzleyen Robot
```

```
## Donanım
```

```
Arduino, IR sensörler, motor sürücü.
```

```
## Başlarken
```

```
Bağlantı şemasını izle.
```

```
Sensör eşiğini kendi koşullarında doğrula.
```

```
Gözlemine ve değişiklik gerekçeni kaydet.
```

Amaç · donanım · bağlantı · çalıştırma · bilinen sınırlar

Herkes görsün mü, herkes yazsın mı?

Görünürlük ile değiştirme yetkisi ayrı kararlardır.

Görmek

Public: herkes görebilir. Private: erişim yetkiyle sınırlıdır.

Değiştirmek

Ana depoya yazmak ayrıca izin gerektirir.

Parolalar ve erişim anahtarları proje dosyası değildir.

Bulduğumuz kodu hemen kullanalım mı?

Bir örnek, ancak donanım ve kullanım koşullarıyla anlamlı.

- Donanım ve ortam uyuyor mu?
- Açıklama, örnek ve bilinen sorunlar var mı?
- Lisans neye izin veriyor?

Yıldız sayısı, robotumuzdaki doğruluğun ölçümü değildir.

Görebildiğimiz kodu kullanabilir miyiz?

Lisans, paylaşımın koşullarını açıklar.

- Kullanım koşullarını oku.
- Gerekli bildirimleri koru.
- Kendi paylaşımının koşullarını açıkça belirt.

Public görünürlük, sınırsız kullanım hakkı anlamına gelmez.

Dış projeye düzeltmeyi nasıl öneririz?

Kendi fork'umuza gönderir, asıl projeye PR açarız.

1. Katkı kılavuzunu oku.
2. Fork → clone → iş dalı → commit → push.
3. Kendi dalından asıl projeye PR öner.

Fork uzak kopyadır. Clone bilgisayardaki kopyadır.

04 · PROJEYİ DEVRET

Bu proje bizi nasıl anlatır?

Dosyaları değil, yaptığımız işi anlaşılır hâle getiriyoruz.

5 FAYDA

GitHub profilin = portfolyon

github.com/elif/robot-projem

README.md

devre-semasi.png

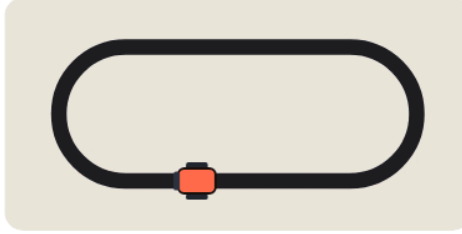
proje-raporu.pdf

robot.ino

README

robot-projem

Çizgi izleyen robot — Mekatronik dersi dönem projesi.



Malzemele

- Arduino U
- 2 DC mot
- 3 kızılöte
- 7.4 V Li-ion p

Nasıl çalışır?

Sensörler siyah çizgiyi görür. Robot çizgiden kaydıkaç motor hızlarını ayarlar ve yola geri döner.

Elif Aydın

Mekatronik Mühendisliği · 3. sınıf

PROJELER

github.com/elif

SENİN İÇİN

Projelerin = portfolyon

✓

Hocalar ve işverenler ne yaptığını görür, sadece okumaz.

✓

Her proje bir vitrin: README'de fotoğraf, şema, açıklama.

✓

Staj ve iş başvurularında linkini CV'ne ekle.

README = projenin vitrini: ne, neden, nasıl.

Staj ya da iş başvurusu? CV'de tek satır yeter: github.com/elif

Sorun · senin katkın · gerekçe · gösterim · bilinen sınırlar

Dersten sonra nereden devam edilir?

İhtiyaç doğduğunda resmî kaynaklara geri dön.

- Kavramlar: Pro Git
- İşlem ayrıntıları: GitHub Docs
- Kendi hızında pratik: GitHub Skills

Kurulum ve komutlar: 12 başvuru slaytı • Öğrenci araçları: GitHub Education

Takımın elinde şimdi ne var?

Artık hangi değişikliği neden paylaştığımızı anlatabiliyoruz.

Başlarken

Farklı “son” dosyalar. Kaybolan gerekçeler. Kimin hangi hâlde olduğu belirsiz.

Şimdi

Eşik kararının gerekçesi PR’da. Güncel main takımın bilgisayarlarında. README projeyi devretmeye hazır.

Git değişikliği kaydeder. Takım, gerekçesiyle karar verir.

Yarın yine çizgiyi kaybederse?

Artık ilk sorumuz belli: “Ne değişti, hangi kanıtla?”

Değişiklik nerede? Kim görüyor? Neden kabul ediyoruz?

[GitHub Docs](#)

[Pro Git • Türkçe](#)

[GitHub Skills](#)

[GitHub Education](#)

Git kurulumu

İşletim sistemine uygun resmî kurulum yolunu seç.

```
git --version
```

Windows

Git for Windows; Git Bash veya uygun terminal.

macOS ve Linux

İşletim sistemine uygun paket veya kurulum yöntemi.

Sürüm numarası sisteme göre değişir. Güncel kurulum: git-scm.com/install

Commit kimliği

Kayıtlarda görünen ad ve e-posta önceden ayarlanır.

```
git config --global user.name "Elif Örnek"  
git config --global user.email "elif@example.com"  
git config --global init.defaultBranch main
```

Ad ve e-posta örnektir. Bu ayar GitHub'da kimlik doğrulama yapmaz.

Editör ve terminal

Aynı Git işlemi terminalde veya grafik arayüzde yapılabilir.

Terminal

Komut ve seçenekleri açıkça görürsün; işlem adımları kolayca paylaşılır.

VS Code

Dosyalar, fark görünümü, Source Control ve entegre terminal aynı pencerededir.

Open Folder ile proje açılır. View → Terminal ile terminal görünür.

.git ve .gitignore

Geçmişin veritabanı ile izleme tercihleri farklı dosyalarda durur.

```
robot-projem/  
  .git/  
  .gitignore  
  README.md  
  robot.ino  
  
# .gitignore örneği  
build/  
.env
```

.git elle düzenlenmez. .gitignore, zaten izlenen dosyaları geçmişten silmez.

SSH anahtar çifti

GitHub'a açık anahtar eklenir. Özel anahtar kişide kalır.

```
ssh-keygen -t ed25519 -C "elif@example.com"
```

```
# GitHub'a eklenecek dosya:
```

```
~/.ssh/id_ed25519.pub
```

```
# Paylaşılmayacak özel anahtar:
```

```
~/.ssh/id_ed25519
```

Anahtar için parola kullanımı koruma sağlar. Mevcut anahtarın üzerine yazılmaz.

SSH ajanı ve bağlantı

Ajanın kurulumu işletim sistemine göre değişir.

```
# Bash / Git Bash örneği  
eval "$(ssh-agent -s)"  
ssh-add ~/.ssh/id_ed25519
```

```
# GitHub bağlantısını kontrol et  
ssh -T git@github.com
```

macOS Keychain ve Windows OpenSSH adımları farklı olabilir. Resmî yönerge izlenir.

Yerel klasörden depoya

Bu örnek, GitHub'da boş bir uzak depo varsayar.

```
cd robot-projem  
git init -b main  
git add README.md robot.ino  
git commit -m "Robot projesinin ilk kaydı"  
git remote add origin <depo-adresi>  
git remote -v
```

README.md ve robot.ino önceden vardır. Uzak depoda başlangıç commit'i varsa clone daha sade olabilir.

İlk gönderim ve izleme

-u, yerel dalın hangi uzak dalı izleyeceğini kaydeder.

```
git push -u origin main
```

```
# İzleme bağlantısı kurulduktan sonra:
```

```
git push
```

```
git pull --ff-only
```

Dal adını git branch ile kontrol et. Komutta projenin gerçek dal adı kullanılır.

Dal ve fark komutları

Karşılaştırmada başlangıç ve sonuç tarafı açık yazılır.

```
git branch
git switch -c feature/sensor-esigi

# İki dalın son hâli: main → feature
git diff main feature/sensor-esigi -- robot.ino

# Ortak başlangıçtan feature dalındaki değişiklik:
git diff main...feature/sensor-esigi
```

Tab tamamlama kabuk ayarına bağlıdır. Yazılan örnek dal adının başlangıcı olmalıdır.

Geri alırken niyeti seç

Dosya, hazırlık alanı ve geçmiş için farklı işlemler vardır.

Hazırlıktan çıkar

`git restore --staged README.md`
Çalışma dosyasındaki değişiklik korunur.

Paylaşılan değişikliği tersine çevir

`git revert <commit>`
Önceki etkiyi geri alan yeni commit oluşturur.

`git reset HEAD~1`, dalı önceki commit'e taşır. Varsayılan mixed çalışma dosyalarını korur.

Hard reset ve kurtarma sınırları

Commit edilmemiş çalışma, hard reset ile kaybolabilir.

```
# Yalnız etkileri değerlendirilmişse:  
git reset --hard <commit>  
  
# Yerel işaretçi hareketlerini incele:  
git reflog
```

Reflog bazı eski commit'leri bulmaya yardımcı olur. Kaydedilmemiş bütün işi kurtarma garantisi vermez.

Komutlara kısa bakış

Komutlar, ana dersteki kavramların kısa karşılığıdır.

```
git clone <depo-adresi>
# Dosyayı düzenle ve kaydet
git status
git add <dosya>
git commit -m "Ne ve neden değişti?"
git push

# Takımın yeniliklerini al
git pull --ff-only
# İş dalı oluştur
git switch -c feature/<is>
```

Yer tutucular gerçek değerlerle değiştirilir. Kurulum, kimlik ve izinler önce sağlanır.